

NLP Task Report

Sentiment Analysis of Tweets using RNN, LSTM, and Word2Vec Embeddings

6CS012

Artificial Intelligence and Machine Learning

Herald College Kathmandu

Student Information:

Name	Student Number	Group	Date
Pratik Karki	2408034	L6CG15	May 10, 2026

Abstract

This project uses a dataset of 20,491 hotel reviews to demonstrate an end-to-end deep learning pipeline for sentiment categorization. Sorting reviews into a 1–5 star rating system was the goal. We developed and compared three architectures: a Simple RNN, a conventional LSTM, and an LSTM leveraging pre-trained glove-wiki-gigaword-50 embeddings. Lemmatization, contraction management, and noise reduction were used as preprocessing techniques for the text. Our results show that the high class imbalance in the dataset (dominance of 4 and 5-star ratings) considerably biases model predictions, even while LSTMs successfully ameliorate the vanishing gradient problem inherent in RNNs. Although the high subjectivity and linguistic subtleties of the review text moderated overall accuracy, the incorporation of pre-trained embeddings accelerated convergence.

Table Of Content

Abstract.....	2
1. Introduction.....	4
Dataset.....	4
Methodology.....	5
Text Preprocessing and Vectorization.....	5
Model Architecture.....	6
Simple RNN Model:.....	6
LSTM Model.....	6
LSTM with Word3Vec Embeddings.....	6
Model Training and Hyperparameters.....	7
Loss Function.....	7
Optimizer.....	7
Hyperparameters.....	7
Experiment and Results.....	7
RNN vs LSTM Performance.....	7
Computational Efficiency.....	8
Training with different Embeddings.....	9
Conclusion.....	12

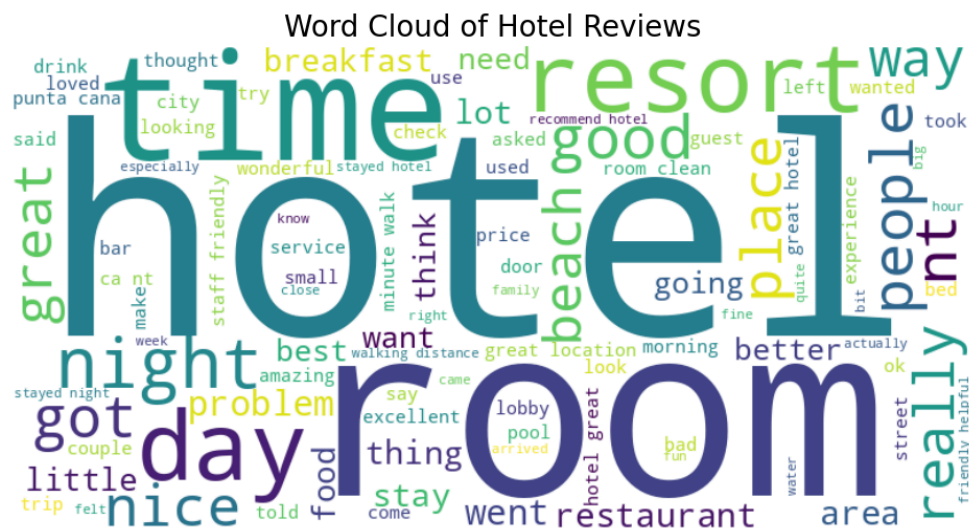
1. Introduction

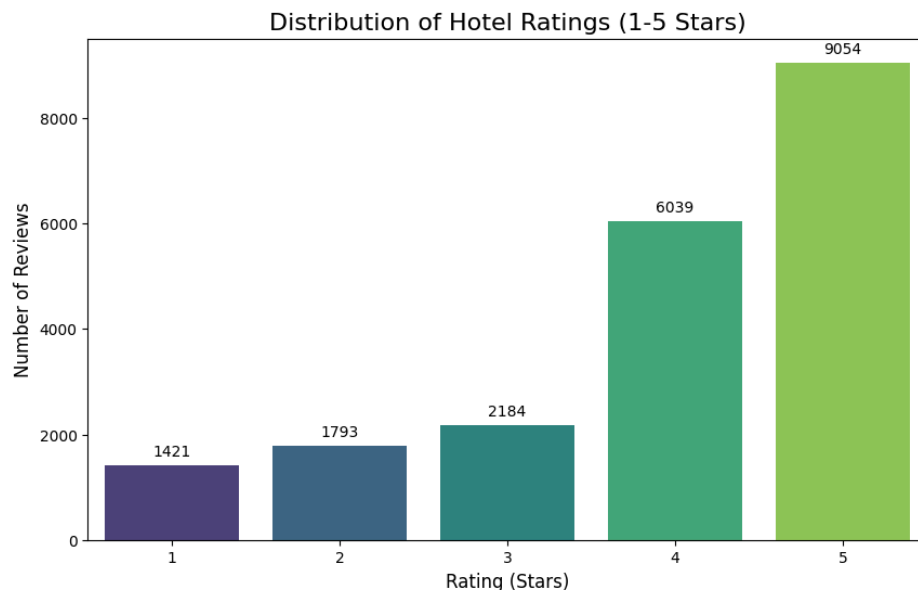
A key component of Natural Language Processing (NLP) is sentiment analysis, which makes it possible to automatically extract emotional tone from text. Accurately categorizing customer evaluations is essential for reputation management in the hospitality sector. Deep learning approaches, such as Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) networks, evaluate text sequentially, enabling the model to "remember" context across sentences, in contrast to typical machine learning models that consider words as independent attributes. This study describes how these models were constructed, assesses how computationally efficient they are on GPU technology, and offers a thorough analysis of how well they perform in classification.

Dataset

In this project, a dataset was provided and classified to a group by tutor. The size of the dataset is approximately 20,491 reviews and the total classes are 5 (1 to 5 star rating). The dataset is a highly imbalanced dataset where the majority of reviews are 4 and 5 stars approximately 15000 and very few 1 and 2 star reviews approximately 3200 total.

For preprocessing, we applied lowercasing, contraction expansion and removal of URLs, special characters. We used the NLTK library to remove stopwords while preserving the words like "not" and WordNetLemmatizer for base word reduction.





Methodology

This project's technique includes a thorough natural language processing (NLP) pipeline that starts with raw text cleaning and ends with a comparative analysis of several recurrent neural network topologies.

Text Preprocessing and Vectorization

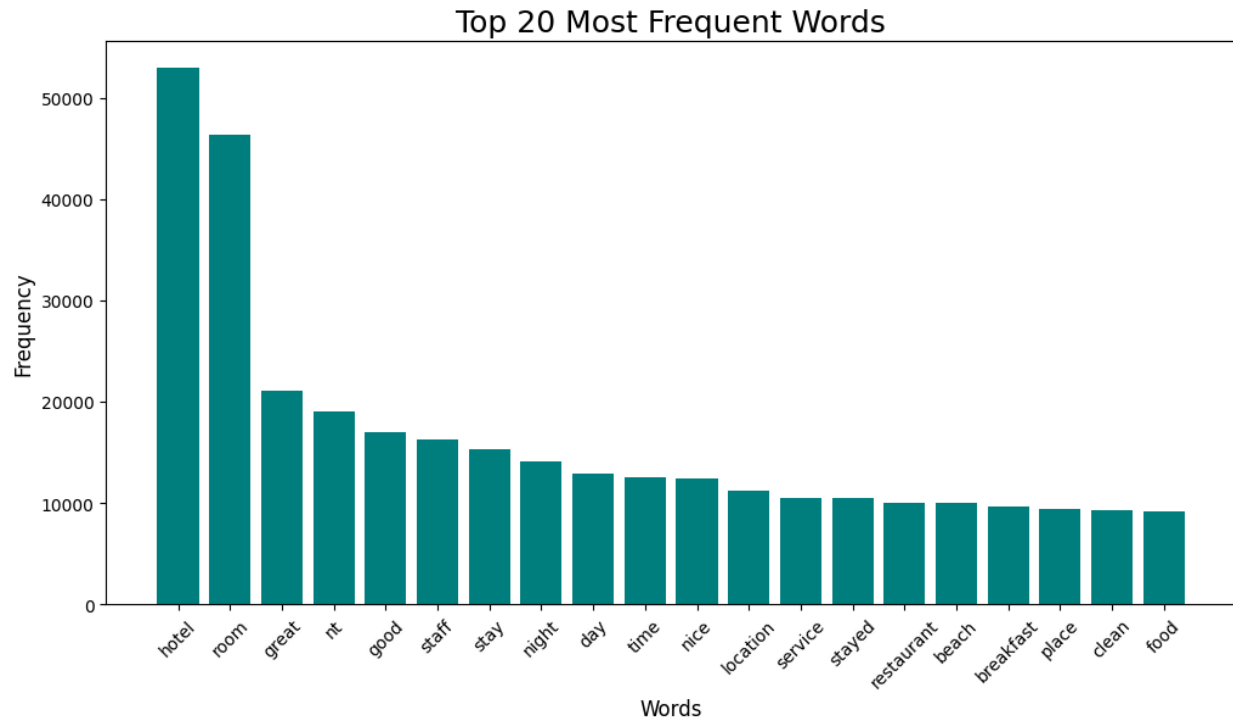
Before training a data, a dataset should be cleaned with various steps. For this hotel review dataset several methods were used such as text cleaning, contraction handling, stopwords removal and lemmatization, tokenization, sequence padding.

First, URLs, hashtags, mentions and special characters were removed using regular expressions. Then, the contractions library was used to expand shortened words like don't to do not, ensuring that the sentiment expressed in negations was preserved.

After, the process is followed by removing stopwords and using lemmatization. Removal of common English stopwords using NLTK library. But a custom filter was made because negative words like not, no, neither are important in sentiment analysis so that it won't be removed while training. Then, lemmatization was done to reduce words to their base form.

Using keras Tokenizer, so that text would convert into sequences of integers, limited to top 20000 unique words.

To ensure uniform input dimensions, the calculated 95th percentile of review lengths, setting maximum sequence length to 264 words. Then, post padding was applied to all sequences.



Model Architecture

In this project 3 different models were analyzed from basic recurrent structures to advanced semantic representations

Simple RNN Model:

The first model serves as a baseline using a Simple Recurrent Neural Network (RNN). It consists of a trainable 128-dimensional embedding layer, a 64-unit SimpleRNN layer, and a fully connected dense layer with ReLU activation. While suitable for sequential data, this model is theoretically susceptible to the "vanishing gradient" problem.

LSTM Model

To address the limitations of the Simple RNN, the second model utilizes an LSTM layer. LSTMs incorporate memory gates (input, forget, and output gates) that allow the network to retain information over much longer sequences. This is particularly effective for hotel reviews where the core sentiment might be found in a specific sentence far from the end of the text.

LSTM with Word3Vec Embeddings

The final model incorporates Transfer Learning by using pre-trained glove-wiki-gigaword-50 embeddings. Instead of learning word relationships from scratch, this model initializes its weights with 50-dimensional vectors trained on billions of words from Wikipedia, providing a

deep semantic understanding of word similarities (e.g., understanding that "excellent" and "superb" are related) from the first epoch.

Model Training and Hyperparameters

In order to obtain a fair comparison between models, all 3 models were trained using same parameters.

Loss Function

As sentiment classification is a classification problem, binary cross-entropy for binary classification and categorical cross-entropy for multiclass classification were used.

Optimizer

The Adam optimizer was used due to its faster convergence, adaptive learning rate and better optimization performance.

Hyperparameters

Hyperparameter	Value
Embedding Dimension	100
Batch size	32
Epochs	10-20
Learning Rate	0.001
RNN/LSTM units	64
Padding length	Percentile-based

Other callback parameters like EarlyStopping, ReduceLROnPlateau and CheckPoint were used to reduce overfitting, to reduce learning rate if overfitting occurs and to save the best model.

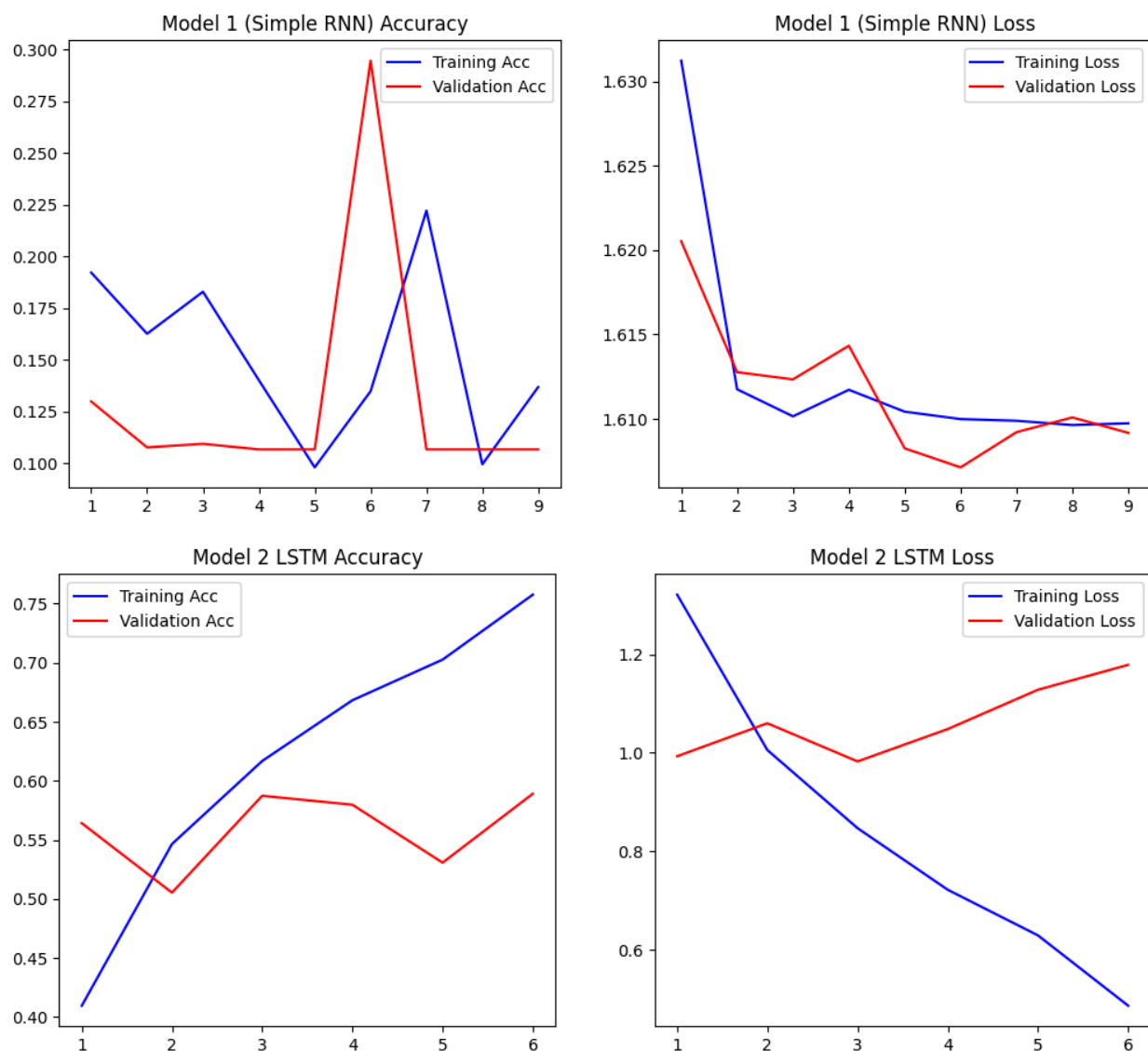
Experiment and Results

In this section, the comparison analysis of models will be detailed.

RNN vs LSTM Performance

The performance of Simple RNN was compared with LSTM. Both models were trained using the same class weights and optimizer, but the actions were different.

The simple RNN model encountered unstable validation loss indicating the issue of Vanishing gradient problem where model failed to link sentiment-heavy words at the beginning of review to final rating. But in another model, LSTM was implemented. Implementing that function made validation accuracy more stable. The gating mechanism of LSTM allowed the model to maintain a cell state, effectively filtering which parts of the hotel review were important to remember and which could be forgotten.



Computational Efficiency

Training was completed using Kaggle notebook on a Tesla P100 GPU.

Metric	Simple RNN	LSTM (Birectional)	LSTM + Word2Vec
Total training time	110 seconds	150 seconds	210 sconds

Time per epoch	9ms/step	51ms/step	63 ms/steps
Complexity	Low (Single Layer)	Moderate (Stacked)	High (Stacked)

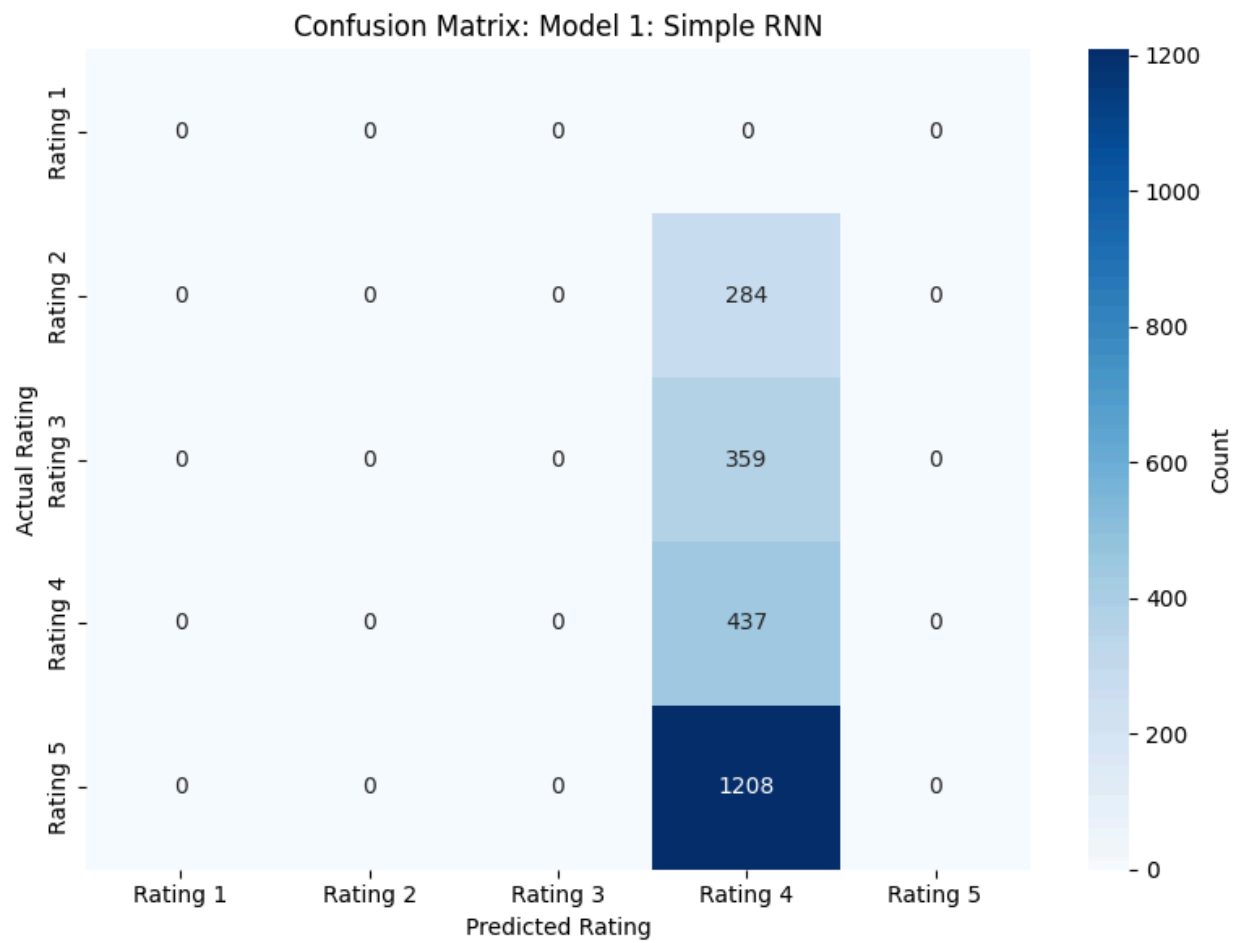
The observation shows that LSTM models were computationally heavier, but they were more efficient in terms of convergence. The LSTM models reached a higher accuracy in 3-5 epochs than the Simple RNN did in 10 which shows better trade-off between time and performance.

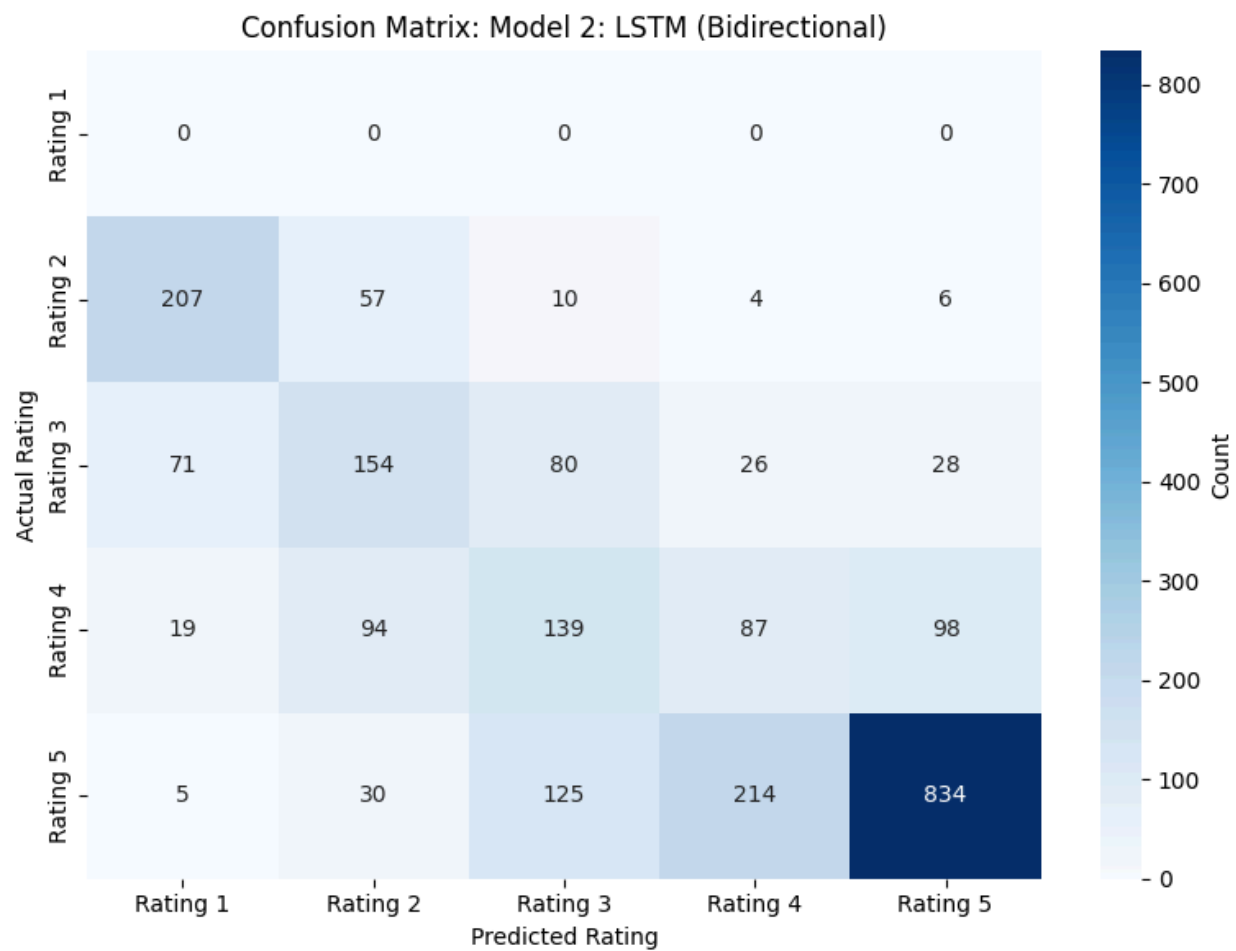
Training with different Embeddings

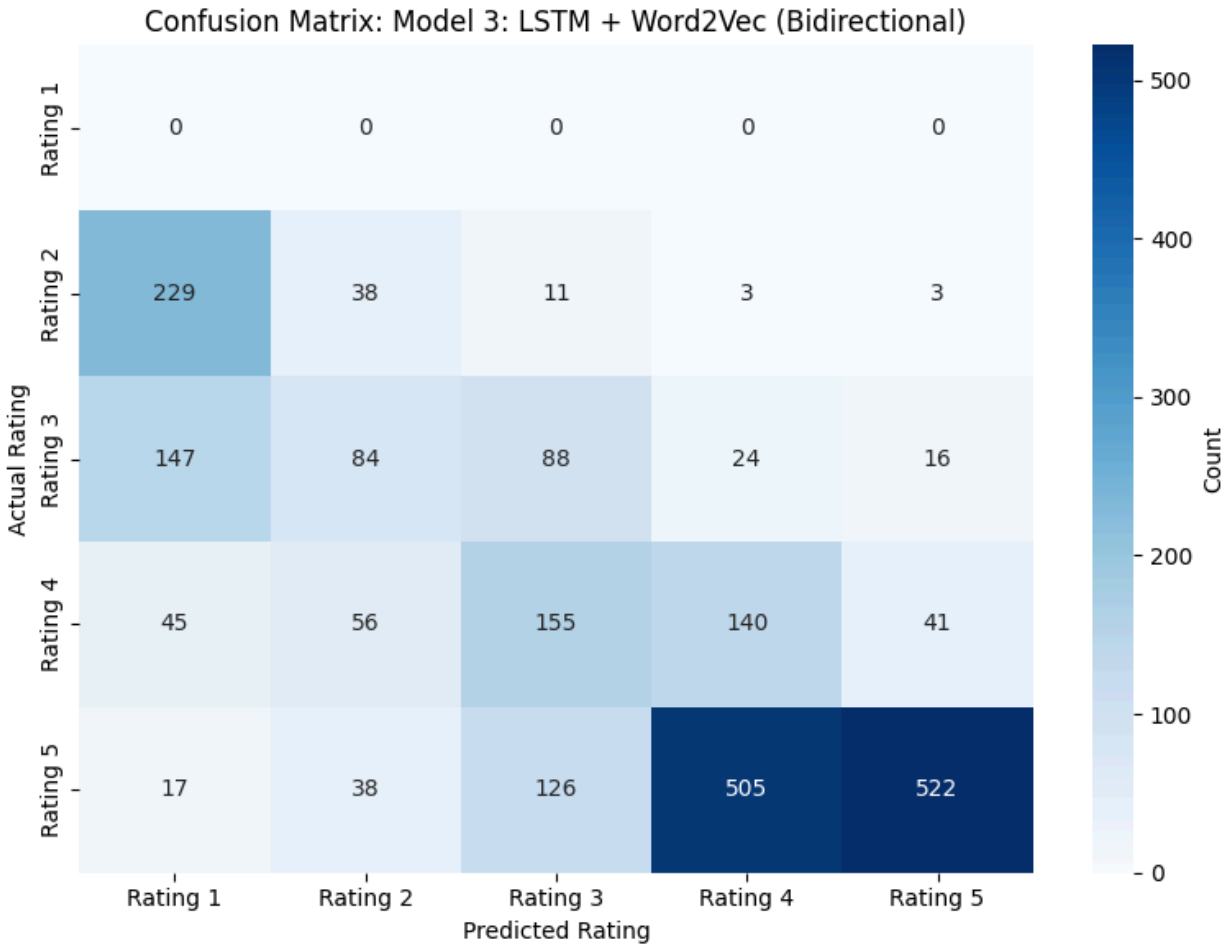
This experiment compared "learning from scratch" vs. "Transfer Learning" with GloVe Twitter 50d .

- **Word2Vec/GloVe Impact:** Model 3 demonstrated significantly faster initial learning. Because the GloVe vectors already grouped similar words (e.g., "amazing" and "wonderful"), the model did not have to spend the first few epochs learning basic English vocabulary.
- **Results:** The pre-trained model achieved a higher initial validation accuracy (~53% in Epoch 1) compared to the trainable embedding model, which started much lower.

Model	Accuracy	Precision	Recall	F1-Score
Simple RNN	29.47%	0.0869	0.2947	0.1342
LSTM	58.79%%	0.5584	0.5879	0.5365
Word2Vec LSTM	58.99%	0.5741	0.5899	0.5759







Conclusion

The main conclusions about the effectiveness of different recurrent neural network architectures for hotel review sentiment classification are summarized in the project's conclusion. The combination of Bidirectional LSTMs with pre-trained GloVe embeddings produced the most reliable results, with an accuracy of 58.99% and a weighted F1-score of 0.5759, according to comprehensive testing done in the Kaggle Notebook. When processing long-form, subjective text sequences, gating mechanisms and semantic transfer learning are essential, as demonstrated by this improved performance above the baseline Simple RNN. The LSTM architectures were able to retain a more stable cell state throughout the 258-word padded sequences, whereas the Simple RNN was a helpful baseline that always struggled with the vanishing gradient problem.

Despite these achievements, a number of significant shortcomings were noted throughout the assessment stage that serve as a basis for further advancements. The biggest obstacle was the dataset's extreme class imbalance, which mathematical class weighting was unable to completely overcome due to the predominance of 5-star evaluations. Additionally, training accuracy greatly exceeded validation performance, indicating that the models started to memorize particular training phrases rather than generalizing sentiment patterns. This is evidence of overfitting. Additionally, the models frequently misclassified highly critical reviews that contained

positive-sounding adjectives, indicating that they had trouble correctly interpreting sarcasm and complex negations.

Future research should focus on data-level balancing methods, like the Synthetic Minority Over-sampling Technique (SMOTE), to give the network a more varied set of negative linguistic patterns in order to overcome these difficulties. Automated hyperparameter tuning could be used to further optimize learning rates and dropout setups. In the end, the most promising extension for this research is switching from conventional recurrent architectures to Transformer-based models like BERT or RoBERTa. Compared to sequential models, these models would probably offer the contextual depth required to resolve the subtleties of human sentiment and sarcasm more successfully since they use self-attention mechanisms to weight word importance worldwide.